Algorytmy wyszukiwania i algorytmy na liczbach

Matura rozszerzona z informatyki – Sesja 6

Kacper Gębusia

Plan sesji

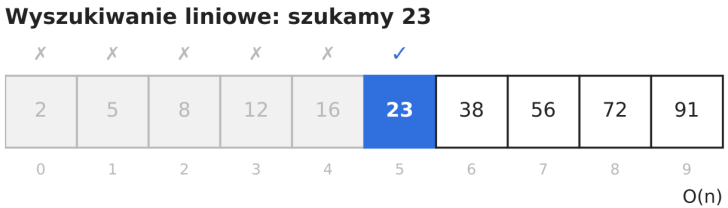
1. Wyszukiwanie liniowe i binarne
2. NWD i NWW – algorytm Euklidesa
3. Liczby pierwsze i sito Eratostenesa
4. Silnia i Fibonacci: rekurencja vs iteracja
5. Ćwiczenia

Powtórka z Sesji 5

1. Wykonaj jeden przebieg sortowania bąbelkowego dla $[3, 1, 4, 2]$.
2. Wykonaj pierwszy krok sortowania przez wybieranie dla $[7, 2, 9, 4]$.
3. Scal listy $[1, 5, 6]$ i $[2, 3, 8]$ i podaj liczbę porównań.
4. Ile porównań wykona sortowanie przez wybieranie dla $n = 8$?

Wyszukiwanie liniowe – idea

Przeglądamy elementy **po kolei**, aż znajdziemy szukany. Działa na **dowolnej** liście.



- złożoność pesymistyczna $O(n)$ (element ostatni lub nieobecny)
- oczekiwana: ok. $\frac{n+1}{2}$ porównań; pamięć $O(1)$
- wbudowane `x in a` i `a.index(x)` też są $O(n)$

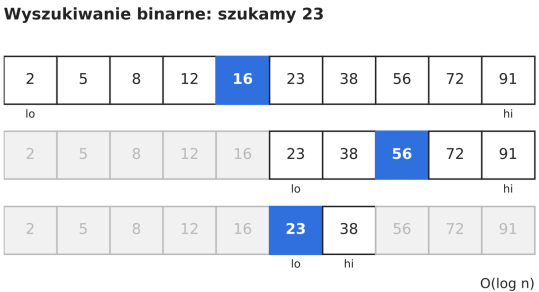
Wyszukiwanie liniowe – kod

```
def szukaj_liniowo(a, x):
    for i in range(len(a)):
        if a[i] == x:
            return i          # indeks znalezionego elementu
    return -1                 # nie znaleziono
```

Wyszukiwanie binarne – idea

Warunek: lista musi być **posortowana**.

Idea: porównujemy szukaną wartość ze środkowym elementem – jeśli jest mniejsza, szukamy dalej tylko w lewej połowie, jeśli większa – w prawej. W każdym kroku odrzucamy połowę pozostałych elementów.



Wyszukiwanie binarne – przykład

$A = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]$ (indeksy 0–9)

Szukamy 23:

`lo`	`hi`	`mid`	`a[mid]`	Decyzja
0	9	4	16	$16 < 23 \rightarrow$ `lo=5`
5	9	7	56	$56 > 23 \rightarrow$ `hi=6`
5	6	5	23	znaleziono: **5**

Szukamy 40:

`lo`	`hi`	`mid`	`a[mid]`	Decyzja
0	9	4	16	$16 < 40 \rightarrow$ `lo=5`
5	9	7	56	$56 > 40 \rightarrow$ `hi=6`
5	6	5	23	$23 < 40 \rightarrow$ `lo=6`
6	6	6	38	$38 < 40 \rightarrow$ `lo=7`
7	6	–	–	`lo>hi`: **brak** (-1)

Wyszukiwanie binarne – kod

```
def szukaj_binarnie(a, x):
    lo, hi = 0, len(a) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if a[mid] == x:
            return mid
        if a[mid] < x:
            lo = mid + 1       # szukamy w prawej polowie
        else:
            hi = mid - 1       # szukamy w lewej polowie
    return -1
```

Wyszukiwanie binarne – złożoność i błędy

- liczba kroków $\leq \lfloor \log_2 n \rfloor + 1$, czyli $O(\log n)$: dla $n = 10$ najwyżej 4 kroki, dla $n = 1000$ – 10, dla $n = 10^6$ – 20; pamięć $O(1)$
- **Kiedy się opłaca?** Sortowanie kosztuje $O(n \log n)$: przy **jednym** wyszukiwaniu wystarczy liniowe, przy **wielu** warto raz posortować i szukać binarnie.

Typowe błędy:

- `lo < hi` zamiast `lo <= hi` (pomija ostatni element)
- `lo = mid` zamiast `mid + 1` (możliwa pętla nieskończona)
- zastosowanie do listy nieposortowanej

NWD i NWW – definicje

NWD (największy wspólny dzielnik) – największa liczba dzieląca obie liczby bez reszty.
NWW (najmniejsza wspólna wielokrotność):

$$\text{NWW}(a, b) = \frac{a \cdot b}{\text{NWD}(a, b)}$$

Metoda naiwna: próbujemy dzielników od $\min(a, b)$ w dół – $O(\min(a, b))$ (wolno).

Liczby, dla których $\text{NWD} = 1$, nazywamy **względnie pierwszymi** (np. 18 i 5).

Euklides przez odejmowanie – idea

$\mathrm{NWD}(a, b) = \mathrm{NWD}(a - b,\ b)$ dla $a > b$; kończymy, gdy liczby się zrównają.

Przykład: $\mathrm{NWD}(48, 18)$:

$$(48, 18) \to (30, 18) \to (12, 18) \to (12, 6) \to (6, 6) \ \Rightarrow\ \mathbf{6}$$

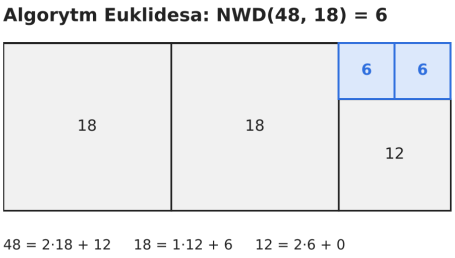
Wada: dla $(1\,000\,000,\ 1)$ wykona milion kroków.

Euklides przez odejmowanie – kod

```
def nwd_odejmowanie(a, b):          # dla liczb dodatnich
    while a != b:
        if a > b:
            a -= b
        else:
            b -= a
    return a
```

Euklides przez resztę – idea

$\mathrm{NWD}(a, b) = \mathrm{NWD}(b,\ a \bmod b)$, oraz $\mathrm{NWD}(a, 0) = a$.



Dlaczego działa: wspólne dzielniki a i b są dokładnie tymi samymi liczbami co wspólne dzielniki b i $a \bmod b$ (bo $a \bmod b = a - k \cdot b$).

Euklides przez resztę – przykłady i złożoność

Przykład: NWD$(48, 18)$: $(48, 18) \to (18, 12) \to (12, 6) \to (6, 0) \Rightarrow$ **6,** więc NWW$(48, 18) = \frac{48 \cdot 18}{6} =$ **144.**

Przykład: NWD$(252, 105)$: $252 \bmod 105 = 42$; $105 \bmod 42 = 21$; $42 \bmod 21 = 0 \Rightarrow$ **21;**
NWW $= \frac{252 \cdot 105}{21} =$ **1260.**

Złożoność: po każdym kroku reszta jest mniejsza niż połowa poprzedniej liczby, więc kroków jest $O(\log \min(a, b))$ – dużo szybciej niż odejmowanie.

Euklides przez resztę – kod

```
def nwd(a, b):
    while b != 0:
        a, b = b, a % b
    return a

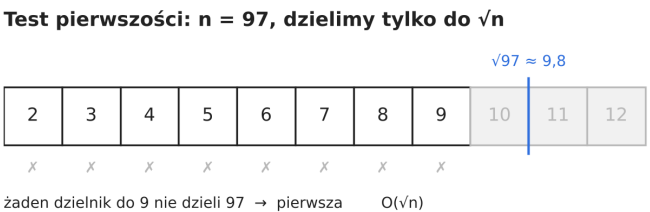
def nwd_rek(a, b):                    # wersja rekurencyjna
    if b == 0:
        return a
    return nwd_rek(b, a % b)

def nww(a, b):
    return a // nwd(a, b) * b
```

Liczby pierwsze – test przez dzielenie

Liczba $n \geq 2$ jest **pierwsza**, jeśli ma dokładnie dwa dzielniki: 1 i n. Liczba 1 **nie** jest pierwsza; 2 jest jedyną parzystą pierwszą.

Obserwacja: jeśli $n = d \cdot e$ i $d \leq e$, to $d \leq \sqrt{n}$. Wystarczy szukać dzielnika tylko do $\sqrt{n}$.



Złożoność $O(\sqrt{n})$ (sprawdzanie do $n - 1$ to $O(n)$).

Liczby pierwsze – przykłady

- $n = 97$: sprawdzamy $i = 2, 3, \ldots, 9$ ($9 \cdot 9 = 81 \leq 97$, $10 \cdot 10 > 97$) – żadna nie dzieli $\Rightarrow$ **pierwsza**
- $n = 91$: dla $i = 7$ mamy $91 = 7 \cdot 13 \Rightarrow$ **nie jest pierwsza**

Liczby pierwsze – kod

```
def czy_pierwsza(n):
    if n < 2:
        return False
    i = 2
    while i * i <= n:
        if n % i == 0:
            return False
        i += 1
    return True
```

Sito Eratostenesa – idea

Gdy potrzebujemy **wszystkich liczb pierwszych do** n: zaczynamy od wszystkich liczb, a dla każdej znalezionej pierwszej **skreślamy jej wielokrotności**.

Sito Eratostenesa: n = 30

1	**2**	**3**	4	**5**	6
7	8	9	10	**11**	12
13	14	15	16	**17**	18
19	20	21	22	**23**	24
25	26	27	28	**29**	30

niebieskie = pierwsze (2,3,5,7,...), skreślamy wielokrotności do √30

- skreślanie zaczynamy od i^2 (mniejsze wielokrotności i skreśliły już mniejsze liczby pierwsze)
- zewnętrzna pętla kończy się na $\sqrt{n}$

Sito Eratostenesa – przykład i złożoność

`sito(30)`:

i	Działanie
2	skreślamy 4, 6, 8, …, 30 (wszystkie parzyste większe od 2)
3	skreślamy 9, 12, 15, …, 30 (nowe: 9, 15, 21, 27)
4	już skreślone – pomijamy
5	skreślamy 25, 30 (nowe: 25)
6	$6 \cdot 6 = 36 > 30 \rightarrow$ koniec

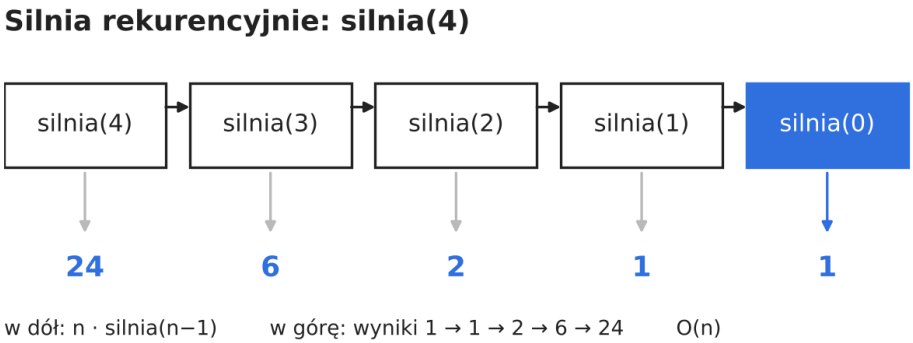
Wynik: **2, 3, 5, 7, 11, 13, 17, 19, 23, 29** (10 liczb).

Złożoność: czasowa $O(n \log \log n)$, pamięciowa $O(n)$. **Wybór:** jedna liczba $\rightarrow$ `czy_pierwsza` ($O(\sqrt{n})$); cały zakres $\rightarrow$ sito. Pierwszych liczb do 100 jest 25, do 1000 – 168.

Sito Eratostenesa – kod

```
def sito(n):
    if n < 2:
        return []
    czy = [True] * (n + 1)
    czy[0] = czy[1] = False
    i = 2
    while i * i <= n:
        if czy[i]:
            for j in range(i * i, n + 1, i):    # skreslamy wielokrotnosci
                czy[j] = False
        i += 1
    return [k for k in range(2, n + 1) if czy[k]]
```

Silnia – idea (powtórzenie z Sesji 4)



Obie wersje (rekurencyjna i iteracyjna) są $O(n)$ czasowo; rekurencyjna zużywa $O(n)$ pamięci (stos), iteracyjna $O(1)$.

Silnia – kod

```
def silnia_rek(n):
    if n == 0:
        return 1
    return n * silnia_rek(n - 1)

def silnia_iter(n):
    wynik = 1
    for i in range(2, n + 1):
        wynik *= i
    return wynik
```

Ciąg Fibonacciego – definicja

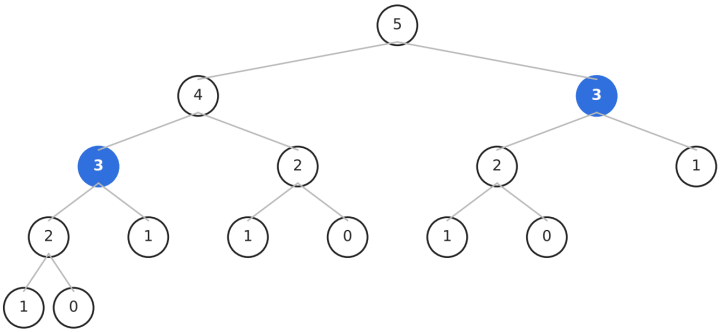
$$F(0) = 0, \quad F(1) = 1, \quad F(n) = F(n-1) + F(n-2)$$

Kolejne wyrazy: $F(0..11) = 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89$.

Fibonacci rekurencyjnie – idea

Problem: te same wartości liczone są **wielokrotnie**. W `fib_rek(5)`: `fib(3)` liczone 2 razy, `fib(2)` – 3 razy, `fib(1)` – 5 razy, `fib(0)` – 3 razy; łącznie 15 wywołań.

fib_rek(5): powtarzane obliczenia (15 wywołań)



fib(3) liczone 2×, fib(2) 3×, fib(1) 5× $O(2^n)$

Fibonacci rekurencyjnie – kod

```
def fib_rek(n):
    if n < 2:
        return n
    return fib_rek(n - 1) + fib_rek(n - 2)
```

Fibonacci rekurencyjnie – liczba wywołań

Liczba wywołań rośnie wykładniczo:

n	5	10	20	30	40
wywołań `fib_rek(n)`	15	177	21 891	2 692 537	331 160 281

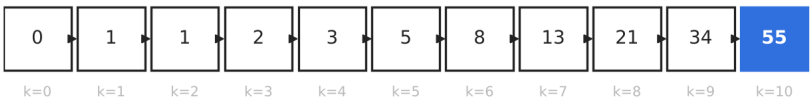
Wzór: liczba wywołań $= 2F(n+1) - 1$.

Złożoność jest **wykładnicza**.

Fibonacci iteracyjnie – idea

Pamiętamy tylko dwie ostatnie wartości (a, b) i przesuwamy je o jeden krok.

Fibonacci iteracyjnie: n = 10



a, b = b, a + b (a, b) = (0,1) → (1,1) → (1,2) → ... → (55,89)

wynik: a = 55

O(n), pamięć O(1)

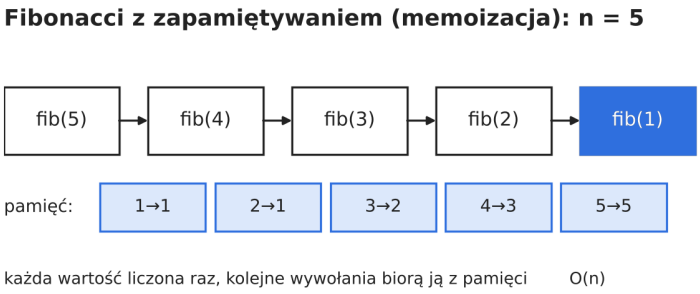
Przebieg dla $n = 10$: $(0, 1) \to (1, 1) \to (1, 2) \to (2, 3) \to (3, 5) \to (5, 8) \to (8, 13) \to (13, 21) \to (21, 34) \to (34, 55) \to (55, 89)$; zwracamy $a = \mathbf{55}$. Złożoność czasowa $O(n)$, pamięciowa $O(1)$.

Fibonacci iteracyjnie – kod

```
def fib_iter(n):
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

Fibonacci z zapamiętywaniem – idea

Memoizacja: zapisujemy obliczone wartości, żeby nie liczyć ich ponownie.



Złożoność $O(n)$ czasowo i $O(n)$ pamięciowo (słownik + stos). W Pythonie to samo daje dekorator `@lru_cache(maxsize=None)` z modułu `functools`. Dla dużych n (rzędu tysiąca) rekurencyjna memoizacja trafi na limit głębokości rekurencji – wtedy wybieramy iterację.

Fibonacci z zapamiętywaniem – kod

```
def fib_memo(n, pamiec=None):
    if pamiec is None:
        pamiec = {}
    if n < 2:
        return n
    if n not in pamiec:
        pamiec[n] = fib_memo(n - 1, pamiec) + fib_memo(n - 2, pamiec)
    return pamiec[n]
```

Rekurencja czy iteracja?

Problem	Rekurencja	Iteracja
Silnia	$O(n)$ czas, $O(n)$ pamięć	$O(n)$ czas, $O(1)$ pamięć
Fibonacci	$O(2^n)$ (naiwnie)	$O(n)$ czas, $O(1)$ pamięć
NWD (Euklides)	$O(\log n)$, krótki zapis	$O(\log n)$, $O(1)$ pamięci

Wniosek: rekurencja jest wygodna, gdy problem ma naturalną strukturę rekurencyjną (podział na mniejsze podproblemy), ale trzeba pilnować, czy te same podproblemy nie są liczone wielokrotnie oraz czy nie przekroczymy limitu głębokości.

Ćwiczenia na zajęciach (1/2)

1. Dla listy $[3, 8, 15, 22, 31, 47, 59, 68, 74, 90]$ prześledź wyszukiwanie binarne dla wartości **47** oraz **50** (podaj kolejne `mid` i wynik).
2. Oblicz NWD i NWW: a) $(84, 36)$ b) $(17, 5)$
3. Które z liczb są pierwsze? Dla złożonych podaj znaleziony dzielnik: $1,\ 2,\ 49,\ 97,\ 221$
4. Użyj sita Eratostenesa dla $n = 50$: wypisz liczby pierwsze i podaj, dla jakich i wykonuje się skreślanie.

Ćwiczenia na zajęciach (2/2)

5. Ile co najwyżej kroków wykona wyszukiwanie binarne w liście miliona elementów?
6. Wypisz pierwsze 12 wyrazów ciągu Fibonacciego ($F(0) \ldots F(11)$).
7. Ile wywołań wykona `fib_rek(6)`? Ile obrotów pętli wykona `fib_iter(6)`?
8. Prześledź NWD(48, 18) algorytmem Euklidesa przez odejmowanie.

Praca domowa (1/3)

1. Prześledź wyszukiwanie binarne w liście $[4, 9, 14, 21, 26, 33, 40, 52, 61, 70, 85]$ dla wartości **61** oraz **30**. Podaj kolejne `mid` i wynik.
2. Oblicz NWD i NWW algorytmem Euklidesa (przez resztę): a) $(1071, 462)$ b) $(48, 180)$
3. Sprawdź, czy liczby 143, 149, 1001 są pierwsze. Dla złożonych podaj dzielnik.

Praca domowa (2/3)

4. Wykonaj sito Eratostenesa dla $n = 60$: wypisz liczby pierwsze i podaj wartości i, dla których skreślamy wielokrotności.
5. Napisz funkcję `ile_pierwszych(a, b)` zwracającą, ile liczb pierwszych jest w przedziale $[a, b]$ (włącznie). Podaj wynik dla $(10, 30)$.
6. Podaj $F(20)$. Ile wywołań wykona `fib_rek(20)`, a ile dodawań `fib_iter(20)`?

Praca domowa (3/3)

7. Napisz `fib_memo` z użyciem słownika i podaj jej złożoność czasową i pamięciową.
8. Porównaj rekurencyjną i iteracyjną implementację Fibonacciego pod względem złożoności czasowej i pamięciowej. Kiedy rekurencja w Pythonie kończy się błędem?
9. Napisz funkcję `nwd_lista(lista)` zwracającą NWD wszystkich elementów listy. Podaj wynik dla `[12, 18, 30]`.

Dziękuję za uwagę!